Day 2: Arrays, Strings & Methods (10-June-2026, Wednesday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 2, learners will be able to:

- Declare, initialize, and traverse one-dimensional and two-dimensional arrays.
- Perform common array operations (searching, sorting, copying).
- Differentiate between `String` immutability and `StringBuilder` mutability.
- Use `String` class methods for manipulation (substring, indexOf, split, etc.).
- Define and invoke methods with parameters and return types.
- Apply method overloading to create flexible APIs.
- Generate logic for pattern printing problems.
- Solve array and string-based LeetCode/HackerRank problems.
- Implement a component of a **Customer Feedback Analysis Engine**.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 1 & Day 2 Overview	15 min
9:15 – 10:00	One-Dimensional Arrays – Declaration, Memory, Operations	45 min
10:00 – 10:30	Two-Dimensional Arrays – Ragged Arrays, Traversal	30 min
10:30 – 10:45	Tea Break	15 min
10:45 – 11:30	String Class – Immutability, Key Methods	45 min
11:30 – 12:00	StringBuilder – Mutability, Performance	30 min
12:00 – 12:30	Hands-on: String vs StringBuilder Benchmark	30 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:15	Methods – Declaration, Parameters, Return, Stack Memory	45 min

2:15 – 3:00	Method Overloading – Rules & Ambiguity	45 min
3:00 – 3:15	Tea Break	15 min
3:15 – 3:45	Pattern Programs – Logic Building	30 min
3:45 – 4:15	Corporate Use Case: Customer Feedback Analysis	30 min
4:15 – 4:30	Coding Problems Walkthrough & Homework	15 min

---

1. One-Dimensional Arrays

1.1 What is an Array?

- Contiguous block of memory storing multiple elements of the **same data type**.
- Index starts from `0` to `length-1`.
- Fixed size after creation (cannot grow or shrink).

1.2 Declaration & Creation

```
// Declaration (no memory allocated)
int[] numbers;          // preferred style
int numbers[];          // C-style, valid but less common

// Memory allocation
numbers = new int[5];   // creates array of 5 ints (default 0)

// Combined declaration + creation
int[] scores = new int[10];

// Array initializer (size inferred)
int[] primes = {2, 3, 5, 7, 11};

// Anonymous array
int[] temp = new int[]{1, 2, 3};
```

1.3 Default Values

Data Type	Default Value
byte, short, int, long	0
float, double	0.0
char	'\u0000' (null character)

boolean	false
Reference (String, Object)	null

1.4 Array Length

- `array.length` (property, not method).
- Valid indices: `0` to `array.length - 1`.

1.5 Traversal & Common Operations

For loop

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Enhanced for-each loop

```
for (int value : arr) {
    System.out.println(value);
}
```

- Limitation: cannot modify array elements (only read).

Common algorithms

- **Linear search** – iterate and compare.
- **Finding min/max** – track current extremum.
- **Array copy** – `System.arraycopy()`, `Arrays.copyOf()`, manual loop.
- **Sorting** – `Arrays.sort()` (Dual-Pivot Quicksort).

1.6 Command-line arguments

- `public static void main(String[] args)` – `args` is a String array.
- Pass values when running: `java MyProgram hello world` → args[0]="hello", args[1]="world".

---

2. Two-Dimensional Arrays

2.1 Representation

- Array of arrays (jagged arrays allowed – rows can have different lengths).

- Declaration: `int[][] matrix;` or `int matrix[][];`
- Creation: `matrix = new int[3][4];` → 3 rows, each row has 4 columns.

2.2 Ragged Arrays

```
int[][] ragged = new int[3][];
ragged[0] = new int[2];
ragged[1] = new int[4];
ragged[2] = new int[3];
```

- Useful for triangular matrices or sparse data.

2.3 Access & Traversal

- Access: `matrix[row][col]`
- Nested loops:

```
for (int i = 0; i < matrix.length; i++) {          // rows
    for (int j = 0; j < matrix[i].length; j++) {   // columns
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

- Enhanced for-each for rows, then columns.

2.4 Common Operations

- **Row-wise sum** – iterate each row, sum columns.
- **Column-wise sum** – ensure all rows have same length (or handle ragged).
- **Transpose** – `result[j][i] = matrix[i][j]`.
- **Matrix multiplication** – triple nested loop.

---

3. String Class

3.1 Immutability

- Once a `String` object is created, its value cannot be changed.
- Any operation that modifies a string creates a **new String object**.
- Example: `String s = "Hello"; s = s.concat(" World");` – original "Hello" remains, new "Hello World" created.
- **Why immutable?**

- Security (strings used in class loading, network connections).
- Thread safety (no synchronization needed).
- String pool optimization (reuse literal strings).

3.2 String Pool (Interning)

- Literal strings (`"abc"`) stored in heap's **string constant pool**.
- Duplicate literals refer to same object.
- `new String("abc")` creates a new object outside pool (unless interned).

3.3 Important String Methods

Method	Description
`length()`	returns number of characters
`charAt(index)`	returns character at index
`substring(begin, end)`	extracts portion (end exclusive)
`indexOf(ch/str)`	first occurrence index, -1 if not found
`lastIndexOf(...)`	last occurrence
`toLowerCase()` / `toUpperCase()`	returns new case-changed string
`trim()`	removes leading/trailing whitespace
`replace(old, new)`	replaces all occurrences
`split(regex)`	splits into String array
`contains(CharSequence)`	boolean check
`startsWith()` / `endsWith()`	prefix/suffix check
`equals()` / `equalsIgnoreCase()`	content comparison (not `==`)
`compareTo()`	lexicographic comparison
`isEmpty()` / `isBlank()` (Java 11+)	checks emptiness or whitespace only

3.4 Concatenation Performance

- Using `+` inside loops is inefficient (creates many intermediate strings).
- Use `StringBuilder` (or `StringBuffer` for thread-safe) for repeated concatenation.

---

4. StringBuilder Class

4.1 Mutability

- `StringBuilder` provides a mutable sequence of characters.
- Operations modify the existing object without creating new ones.
- Preferred for heavy string manipulation (loops, building dynamic strings).

4.2 Key Methods

Method	Effect
`append(String/char/int/... )`	adds at the end
`insert(index, value)`	inserts at position
`delete(start, end)`	removes characters
`deleteCharAt(index)`	removes single character
`replace(start, end, str)`	replaces range
`reverse()`	reverses the sequence
`toString()`	converts to immutable String

4.3 Capacity & Performance

- `StringBuilder` maintains an internal char array.
- `capacity()` – current allocated size (not the length).
- If length exceeds capacity, it expands (typically doubles + 2).
- Use `ensureCapacity()` to pre-allocate and avoid reallocations.

4.4 String vs StringBuilder – When to Use?

Scenario	Choice
Fixed string, few concatenations	String
Building string in loop (e.g., 100+ iterations)	StringBuilder
String manipulation with many modifications	StringBuilder
Need thread-safety across multiple threads	StringBuffer (slower, synchronized)
Using string as key in HashMap	String (immutable guarantees hash code stability)

	stability)

5. Methods in Java

5.1 Method Declaration

```
[accessModifier] [static] returnType methodName(parameterList) {
    // method body
    [return value;]
}
```

- **Access modifiers:** `public`, `protected`, default (package-private), `private`.
- **Return type:** primitive, reference, or `void` (no return).
- **Parameters:** zero or more, each with type and name.

5.2 Calling a Method

- **Static method:** `ClassName.methodName(args)` or within same class just `methodName(args)`.
- **Instance method:** `objectReference.methodName(args)`.

5.3 Pass by Value

- Java is **strictly pass-by-value**:
  - Primitive types – the value is copied. Changes inside method do not affect original.
  - Reference types – the reference value (memory address) is copied. Modifying object state affects original, but reassigning the reference does not.
- Example:

```
void changePrimitive(int x) { x = 10; }           // no change to caller
void changeObject(Person p) { p.name = "John"; }   // changes original object
void reassignObject(Person p) { p = new Person(); } // no effect on caller
```

5.4 Method Stack Memory

- Each method call creates a **stack frame** containing:
  - Local variables
  - Parameters
  - Return address
- When method returns, its frame is popped.
- Recursive methods consume stack; deep recursion may cause `StackOverflowError`.

5.5 Variable Arguments (varargs)

- Syntax: `returnType methodName(Type... varName)`
- Allows zero or more arguments of that type.
- Internally treated as array.
- Must be the last parameter.

---

6. Method Overloading

6.1 Definition

- Multiple methods in the **same class** with the **same name** but **different parameter lists**.
- Return type **alone** is not sufficient for overloading.
- Compiler resolves which method to call based on argument types at compile-time (static polymorphism).

6.2 Rules for Overloading

- Parameter count differs, **or**
- Parameter types differ (including order), **or**
- Type promotion (widening) may be used if exact match not found.

6.3 Valid vs Invalid Overloading

Methods	Valid?	Reason
`void print(int a)` and `void print(double b)`	✅ Yes	Different parameter types
`int add(int a, int b)` and `double add(int a, int b)`	❌ No	Same parameter list
`void show(String s)` and `void show(String s, int n)`	✅ Yes	Different number of parameters
`void work(int a, float b)` and `void work(float a, int b)`	✅ Yes	Different order of types

6.4 Overloading and Type Promotion

- If exact match not found, Java looks for next **widening** conversion (byte→short→int→long→float→double).
- Boxing/unboxing (int↔Integer) and varargs are considered after widening.
- Ambiguous calls cause compile error:

```
void method(int a, float b) { }
void method(float a, int b) { }
method(1, 2); // error: both methods applicable
```

6.5 Practical Use Cases

- `System.out.println()` – overloaded for all primitive types and objects.
- `Math.max()` – overloaded for int, long, float, double.
- Constructor overloading (discussed Day 3).

---

7. Pattern Programs (Logic Building)

7.1 Why Patterns?

- Train logical thinking and nested loop control.
- Understand row-column relationships.
- Prepare for coding interviews (frequently asked).

7.2 Common Pattern Types

Pattern Type	Example (n=4)	Logic Summary
Square	`****` `****` `****` `****`	Outer loop rows, inner loop columns, both 1 to n.
Right triangle (increasing stars)	`*` `**` `***` `****`	Row i has i stars.
Right triangle (decreasing stars)	`****` `***` `**` `*`	Row i has n-i+1 stars.
Pyramid (centered)	`   *` `  ***` `*****` `*******`	Spaces = n-i, stars = 2i-1.
Inverted pyramid	`*******` `  *****` `   ***` `    *`	Spaces = i-1, stars = 2(n-i)+1.
Diamond	Combination of pyramid + inverted	Split into upper and lower halves.
Number patterns	`1` `12` `123` `1234`	Print j instead of star.
Floyd's triangle	`1` `2 3` `4 5 6` `7 8 9 10`	Increment a counter per print.

7.3 Approach to Solve Any Pattern

1. Determine number of rows (usually n).
2. For each row, determine:
    - Number of leading spaces (if any).
    - Number of characters/stars/numbers.
    - Whether to print additional symbols (e.g., `*` and `#` alternating).
3. Write outer loop for rows.
4. Inner loop(s) for spaces, then inner loop for symbols.
5. Use `System.out.print()` for same line, `println()` after each row.

7.4 Common Pitfalls

- Off-by-one errors in loop bounds.
- Forgetting to reset printed characters count.
- Using `println` inside inner loop.

---

8. LeetCode & HackerRank Problems – Approach Discussion

8.1 LeetCode: Two Sum

- **Problem:** Given array nums and target, return indices of two numbers that add to target.
- **Brute force:** Nested loops → $O(n^2)$.
- **Optimal:** Use HashMap to store complement (target – current) and its index → O(n) time, O(n) space.
- **Key takeaway:** HashMap lookups enable linear solution.

8.2 LeetCode: Maximum Subarray (Kadane's Algorithm)

- **Problem:** Find contiguous subarray with largest sum.
- **Kadane's algorithm:**
    - `currentSum = max(currentSum + nums[i], nums[i])`
    - `maxSum = max(maxSum, currentSum)`
- **Edge cases:** All negative numbers – algorithm works (will pick least negative).
- **Time complexity:** O(n), space O(1).

8.3 LeetCode: Valid Anagram

- **Problem:** Two strings s and t, check if t is anagram of s (same characters with same frequency).
- **Approach 1:** Sort both strings, compare equality → O(n log n).

- **Approach 2:** Count frequency of each character using array of size 26 → O(n).
- **Key insight:** Unicode handling? For extended chars, use HashMap.

8.4 HackerRank: Java Arrays

- **Task:** Read integers, perform operations (reverse, find max, etc.).
- **Skills practiced:** Array creation, looping, using `Arrays.toString()` for printing.

8.5 HackerRank: Java Strings Introduction

- **Task:** Read two strings, compute length sum, compare lexicographically, capitalize first letter.
- **Methods used:** `length()`, `compareTo()`, `substring(0,1).toUpperCase() + rest`.

---

9. Corporate Use Case: Customer Feedback Analysis Engine

9.1 Problem Context

A retail company collects thousands of customer feedback comments daily. They need a system to:

- Store multiple feedback strings.
- Analyze frequency of certain keywords (e.g., "good", "bad", "slow").
- Find the longest feedback (for moderation).
- Detect potential spam (multiple identical feedbacks).
- Generate a summary report: total characters, average length, most common word length.

9.2 Application of Day 2 Topics

Requirement	Java Feature Used
Store list of feedbacks	1D String array or ArrayList (Day 6 covers ArrayList; for Day 2, simple array)
Calculate total characters across all feedbacks	Loop through array, `String.length()`
Find longest feedback	Track max length and corresponding string
Count keyword occurrences	`String.indexOf()` in loop, or `split()` + counting
Detect duplicate feedbacks	Nested loops compare strings using `equals()`
Capitalize each feedback for report	`substring(0,1).toUpperCase() + substring(1)`

Build a formatted report string efficiently	`StringBuilder` with `append()`
Process feedbacks row by row	For-each loop or indexed loop
Compare two feedbacks ignoring case	`equalsIgnoreCase()`
Split feedback into words	`split("\\s+")` (regex for whitespace)

9.3 Example Workflow (Conceptual)

1. **Input:** Three feedbacks:
   - "Good product, fast delivery"
   - "Bad customer service"
   - "Good product, fast delivery" (duplicate)
2. **Store** in String array.
3. **Analyze:**
   - Total characters = 49
   - Longest = "Good product, fast delivery" (26 chars)
   - Keyword "good" appears 2 times (case-insensitive)
   - Duplicate detected: feedback 0 and 2 equal.
4. **Generate report** using `StringBuilder`:

```
=== Feedback Summary ===
Total feedbacks: 3
Total characters: 49
Average length: 16.33
Most common word length: 4 (word "good")
Duplicate found? Yes
```

5. **Output** to console.

9.4 Discussion Questions

- How would you ignore punctuation when counting words? (Use `replaceAll("[^a-zA-Z ]", "")`)
- What if feedbacks are too many for a simple array? (Need dynamic structure – ArrayList, Day 6)
- How to make the keyword search case-insensitive? (`toLowerCase()` on both)
- Why use `StringBuilder` instead of `+` for building the report? (Multiple concatenations inside loop)

---

10. Hands-on Exercises (No Code Lines – Logic & Worksheets)

10.1 Array Tracing

Given the following operations, determine final array content (pen and paper):

- `int[] arr = {3, 1, 4, 1, 5};`
- Loop i from 0 to length-2: `arr[i] = arr[i+1];`
- What is the new array? Which element is lost?

10.2 2D Array – Sum of Diagonal

- Describe the condition to identify elements on the main diagonal of a square matrix.
- Describe the condition for the anti-diagonal.
- If matrix is 4×4, which indices belong to both diagonals?

10.3 String Method Predictions

Without running, predict output:

- `"hello".substring(1, 3)`
- `"abcabc".replace('a', 'x')`
- `"  spaces  ".trim().length()`
- `"Java".compareTo("JavaScript")` (positive or negative?)

10.4 Method Overloading – Which is Called?

Given overloaded methods:

```
void test(int a, double b) { }
void test(double a, int b) { }
void test(int a, int b) { }
```

What is invoked for:

- `test(5, 10)`
- `test(5, 10.5)`
- `test(5.5, 10)`

10.5 Pattern Logic Description

Describe (in English steps) how to print:

```
1
```

```
2 3
4 5 6
7 8 9 10
```

(Hint: Floyd's triangle – maintain a counter outside inner loop)

---

11. Quiz & Quick Check (Conceptual)

1. What is the output of: `int[] a = {1,2,3}; int[] b = a; b[0]=5; System.out.println(a[0]);`

- (a) 1
- (b) 5
- (c) Compilation error
- (d) NullPointerException

2. Which statement about `StringBuilder` is true?

- (a) It is immutable.
- (b) It is synchronized (thread-safe).
- (c) `append()` returns a reference to the same object.
- (d) It cannot be converted to String.

3. Given `String s = "Java"; s.concat(" Rocks"); System.out.println(s);` What prints?

- (a) Java Rocks
- (b) Java
- (c) Rocks
- (d) null

4. Which of these is a valid method overload?

- (a) `void show(int x)` and `int show(int x)`
- (b) `void display(String s)` and `void display(String s, int n)`
- (c) `void compute(int a, float b)` and `int compute(float a, int b)`
- (d) Both b and c

5. For a 2D ragged array `int[][] arr = new int[3][]; arr[0]=new int[2]; arr[1]=new int[4]; arr[2]=new int[3];` How many total integers can be stored?

- (a) 9
- (b) 12
- (c) 8

- (d) 10

Answers: 1-b, 2-c, 3-b, 4-d (b and c both valid, a is invalid return type only), 5-a (2+4+3=9)

---

12. Group Activity – Pair Programming Simulation

- **Duration:** 20 minutes (during hands-on slot).
- **Task:** One driver, one navigator. Write pseudocode/algorithm for:
  - **Customer Feedback Analysis** – given an array of feedback strings, find the most frequent word across all feedbacks (ignoring case and punctuation).
  - Steps to discuss:
    a. Extract words from each feedback (split by spaces).
    b. Clean words (remove punctuation, to lowercase).
    c. Count frequency (using a map – preview of Day 7).
    d. Find max count.
- **Outcome:** Each pair explains their approach. Instructor highlights use of `String.split()` and `HashMap` (brief intro).

---

13. Homework & Preparation for Day 3

- **Coding practice (HackerRank):**
  - *Java Arrays* – solve at least 2 variations.
  - *Java Strings Introduction* – complete all test cases.
- **LeetCode:**
  - *Two Sum* (implement both brute force and HashMap solution).
  - *Maximum Subarray* (implement Kadane's algorithm).
  - *Valid Anagram* (implement frequency array method).
- **Pattern practice:** Print the following for n=5 using nested loops (without actually coding, write step-by-step logic):

```
    *
   ***
  *****
 *******
*********
```

- **Reading for Day 3:** OOP Fundamentals – Classes & Objects, Constructors, `this`, `static`, Encapsulation.
- **Self-assessment:**
  - Write a method that takes a 2D array and returns its transpose.

- Explain in your own words why `String` is immutable and why that matters.

---

14. Additional Resources & Cheat Sheets

- **Array quick reference:** `int[] arr = new int[size]; arr.length`; use `java.util.Arrays` class (`toString()`, `sort()`, `binarySearch()`, `copyOf()`).
- **String API documentation:** String (Java 17)
- **StringBuilder vs StringBuffer** – difference table.
- **Method overloading rules** – mnemonic: "Same name, different args; return type doesn't play."

---

15. Closing Summary

Day 2 built upon Day 1's fundamentals by introducing structured data storage (arrays) and text processing (String/StringBuilder). Learners explored methods as reusable blocks and overloaded them for flexibility. Pattern programming honed nested loop logic, essential for interview coding tests. The corporate case study of a Customer Feedback Analysis Engine tied everything together: storing multiple strings, analyzing content, and generating a report using efficient string manipulation.

Key Takeaways:

- Arrays are fixed-size, zero-indexed containers.
- 2D arrays are arrays of arrays (jagged allowed).
- `String` is immutable; `StringBuilder` is mutable and faster for loops.
- Methods encapsulate behavior; overloading provides multiple entry points.
- Patterns train loop reasoning – a core skill for algorithmic thinking.